PRÁCTICA 1

UAH

Doble Grado en INFOADE

Curso 25/26

Edgar Alexis Conforme Vacacela

Jorge González Renedo

Óscar Morán Alarcón

INDICE

INDICE 1
INTRODUCCIÓN 2
PARTE 1: PLANIFICACIÓN CLÁSICA CON PDDL 3
EJERCICIO 1.1: LOGÍSTICA DE SERVICIO DE EMERGENCIAS, VERSIÓN INICIAL 3
Problema 1: 1 persona y 1 caja 4
Problema 2: 2 personas y 3 cajas 4
EJERCICIO 1.2: Generador de problemas en Python 4
Desarrollo del Generador de Problemas 5
Metodología de Pruebas (Benchmark) 7
Análisis de Resultados y Rendimiento del Planificador FF 7
EJERCICIO 1.3: Comparativa rendimiento algoritmos de búsqueda y heurísticas 8
Primer Apartado: 9
Segundo Apartado 10
Tercer Apartado 10
PARTE 2: PLANIFICACIÓN CON NÚMEROS Y OPTIMIZACIÓN 11
EJERCICIO 2.1: SERVICIO DE EMERGENCIAS, TRANSPORTADORES 12
EJERCICIO 2.2: SERVICIO DE EMERGENCIAS, COSTES DE ACCIÓN 14
Estudio de resultados para planificadores de satisfacción 16
Estudio de resultados para planificadores de optimización 17
PARTE 3: PLANIFICACIÓN CON TIEMPOS 18
CONCLUSIONES 20

INTRODUCCIÓN

Este trabajo documenta el diseño y evaluación de un sistema de planificación automática orientado a la logística de emergencias con drones, utilizando el estándar PDDL. A través de tres fases, se evoluciona desde un modelo clásico STRIPS para entregas básicas, pasando por la gestión de costes numéricos y optimización de recursos como transportadores, hasta alcanzar un modelo de planificación temporal que permite la ejecución de acciones concurrentes para minimizar el tiempo total de respuesta.

PARTE 1: PLANIFICACIÓN CLÁSICA CON PDDL

EJERCICIO 1.1: LOGÍSTICA DE SERVICIO DE EMERGENCIAS, VERSIÓN INICIAL

Para la realización de este ejercicio se ha desarrollado empleando el estándar PDDL bajo el nivel de expresividad de STRIPS. Para el dominio se han definido:

Los siguientes objetos encargados de representar la realidad del problema.

- location
- box
- bcontent (Para el contenido de la caja)
- grip (Para los brazos del dron)
- person
- dron

Los siguientes predicados para controlar el funcionamiento de los objetos. Se pueden dividir en 3 tipos:

- Control de localización: (*at-dron ?d - dron ?l - location*), (*at-box ?b - box ?l - location*) y (*at-person ?p - person ?l - location*)
- Control de contenido: (*box-has ?b - box ?c - bcontent*) y (*person-has ?p - person ?c - bcontent*)
- Control de agarre del dron: (*free ?g - grip)* y *(*carrying ?d - dron ?g - *grip ?b - box*)

Las siguientes acciones que se pueden realizar.

- Move: Acción encargada de mover el dron de una posición a otra, simpre y cuando el dron se encuentre en la posición indicada por el primer parámetro.
- Pick: Acción encargada de agarrar la una caja, siempre y cuando se encuentre en la misma localización que el dron y que el brazo de este esté libre.

- Leave: Acción encargada de entregar la caja a una persona (liberando el brazo), siempre y cuando la caja transportada por el dron contenga el contenido deseado por el individuo y estos se encuentren en la misma localización.

Una vez se ha definido el dominio, se procede a definir dos problemas para verificar el funcionamiento del mismo. El primero de ellos constará de una única persona y una sola caja, mientras que el segundo de ellos contará con dos personas y tres cajas.

Problema 1: 1 persona y 1 caja

Para este problema se define como estado inicial una situación en la que hay dos localizaciones, el depósito (donde se encuentran el dron y la caja) y el refugio (donde se encuentra la persona). Además de ello, el dron tiene ambos brazos liberados y el contenido de la caja es comida. La meta será que la persona reciba una caja con comida.

Tras ejecutar el problema, obtenemos el plan mostrado en la figura 1.1, por lo que se puede afirmar que la ejecución es correcta.

```
problememergencias1.pddl.plan
1   ( PICK BOX1 DEPOSITO DERECHA DRON1 )
2   ( MOVE DEPOSITO REFUGIO DRON1 )
3   ( LEAVE BOX1 REFUGIO DERECHA DRON1 PERSON1 COMIDA )
```

Figura 1.1.1 Plan resultado obtenido del problema 1

Problema 2: 2 personas y 3 cajas

De forma muy similar a la anterior, el estado inicial cuenta con las localizaciones depósito (donde se encuentran el dron y las cajas) y refugio (donde se encuentran las personas). El dron también tiene ambos brazos liberados, y en esta ocasión la caja 1 y 2 contienen comida y la caja 3 contiene medicina. La meta será que la persona 1 tenga comida y la persona 2 medicina.

Tras ejecutar el problema, obtenemos el plan mostrado en la figura 1.2, por lo que se puede afirmar que la ejecución es correcta.

```
problememergencias2.pddl.plan
1   ( PICK BOX2 DEPOSITO DERECHA DRON1 )
2   ( PICK BOX3 DEPOSITO IZQUIERDA DRON1 )
3   ( MOVE DEPOSITO REFUGIO DRON1 )
4   ( LEAVE BOX2 REFUGIO DERECHA DRON1 PERSON1 COMIDA )
5   ( LEAVE BOX3 REFUGIO IZQUIERDA DRON1 PERSON2 MEDICINA )
```

Figura 1.1.2 Plan resultado obtenido del problema 2

EJERCICIO 1.2: Generador de problemas en Python

Desarrollo del Generador de Problemas

Para llevar a cabo el análisis de escalabilidad de los planificadores, se ha partido del script en Python proporcionado por el profesor. El objetivo de esta herramienta es automatizar la creación de instancias de problemas en formato PDDL, garantizando que su estructura sea compatible con el dominio de emergencias diseñado en el apartado anterior.

A partir del código base, se han implementado diversas adaptaciones para mapear la lógica genérica a nuestra definición concreta de tipos y predicados. Además, de cara a facilitar la experimentación posterior del benchmark, se modificó la entrada de parámetros original (basada en argumentos de terminal) por una entrada simplificada que solicita un único tamaño n. Este valor n define simétricamente el número de localizaciones, personas, cajas y metas (l=p=c=g=n), manteniendo fijo el número de drones a 1 y sin transportadores, tal y como dictan las restricciones de este primer ejercicio.

La finalización del script se ha centrado en tres secciones fundamentales del archivo PDDL generado:

Definición de Objetos (`:objects`) Se ha sustituido la impresión genérica de objetos del esqueleto para adaptarla a la jerarquía de tipos de nuestro dominio (`dron`, `location`, `box`, `bcontent`, `person`, `grip`). Un aspecto clave añadido en esta sección es la instanciación dinámica de los brazos del dron. Por cada dron generado, el script añade automáticamente dos objetos de tipo `grip` (por ejemplo, `dron1-izq` y `dron1-der`), necesarios para controlar la capacidad de carga simultánea.

```
problem_name = f"drone_problem_d{options.drones}_l{options.locations}_p{options.persons}_c{options.crates}_g{options.go

with open(problem_name + ".pddl", 'w') as f:
    f.write(f"(define (problem {problem_name})\n")
    f.write("(:domain emergencias)\n")
    f.write("(:objects\n")

    # --- COMPLETADO: Escritura de objetos según tu PDDL ---
    f.write("\t" + " ".join(drone) + " - dron\n")
    f.write("\t" + " ".join(location) + " - location\n")
    f.write("\t" + " ".join(crate) + " - box\n")
    f.write("\t" + " ".join(content_types) + " - bcontent\n")
    f.write("\t" + " ".join(person) + " - person\n")
    f.write("\t" + " ".join(grip) + " - grip\n")
    f.write(")\n\n")

    # --- COMPLETADO: Generación del estado inicial ---
    f.write("(:init\n")
```

Figura 1.2.1 Estructuración del esqueleto

Construcción del Estado Inicial (`:init`) En el espacio reservado del esqueleto para el estado inicial, se ha programado la lógica que posiciona los elementos en el mundo. El estado inicial se genera combinando factores deterministas y aleatorios:

- Determinista: Todos los drones se ubican inicialmente en la localización `deposito` y sus brazos (`dronX-izq`, `dronX-der`) se marcan como libres mediante el predicado `(free ?g)`. Las cajas generadas también se instancian inicialmente en el depósito.
- Aleatorio: Se asigna el contenido (comida o medicina) a cada caja siguiendo la distribución calculada por las funciones de ayuda del script (`crates_with_contents`). Asimismo, las personas se distribuyen aleatoriamente escogiendo una localización cualquiera que no sea el depósito base (los refugios).

```
# Generación del estado inicial ---
f.write("(:init\n")

# Drones en el deposito
for d in drone:
    f.write(f"\t(at-dron {d} deposito)\n")

# Brazos libres
for g in grip:
    # Asumiendo tu versión original de dominio: (free ?g)
    f.write(f"\t(free {g})\n")

# Cajas en deposito y su contenido
for content_index, crate_list in enumerate(crates_with_contents):
    for c in crate_list:
        f.write(f"\t(at-box {c} deposito)\n")
        f.write(f"\t(box-has {c} {content_types[content_index]})\n")

# Personas aleatorias en localizaciones (no deposito)
for p in person:
    rand_loc = random.choice(location[1:]) # Omite el deposito
    f.write(f"\t(at-person {p} {rand_loc})\n")

f.write(")\n\n")
```

Figura 1.2.2 Asignación de valores

Definición de Metas (`:goal`) Para definir el estado objetivo, el script hace uso de la matriz de necesidades `need` generada en fases previas, la cual indica qué persona requiere qué contenido específico. Se ha completado el bucle iterativo para traducir esta matriz en sentencias PDDL utilizando nuestro predicado `(person-has ?p ?c)`.

Cabe destacar que en el esqueleto original se sugería la posibilidad de obligar al dron a volver al depósito tras finalizar las entregas. Sin embargo, al analizar detenidamente los requisitos del enunciado, se ha optado por prescindir de esta restricción en el `:goal`, ya que el objetivo primordial del problema es exclusivamente satisfacer la demanda de suministros de los heridos, otorgando así mayor libertad al planificador a la hora de encontrar planes más cortos.

```python
# Metas ---
f.write("(:goal (and\n")

# Necesidades satisfechas
for x in range(options.persons):
    for y in range(len(content_types)):
        if need[x][y]:
            f.write(f"\t(person-has {person[x]} {content_types[y]})\n")
```

Figura 1.2.3 Establecer átomos de la meta

Como resultado de esta implementación, el script es capaz de generar archivos PDDL sintácticamente correctos, estandarizados y preparados para ser resueltos de forma masiva por el planificador automático.

Metodología de Pruebas (Benchmark)

Para evaluar la escalabilidad del dominio y el rendimiento de los planificadores, se desarrolló un script automatizado en Python (benchmark_grafica.py). Este script actúa como un orquestador que ejecuta iterativamente el generador de problemas previamente descrito, incrementando el tamaño del problema n (donde n es el número de localizaciones, personas, cajas y metas simultáneamente) en una unidad por cada iteración, comenzando desde n=1.

Por cada problema generado, el orquestador invoca de forma secuencial al planificador automático FF (Fast Forward) y registra el tiempo en segundos que este tarda en encontrar un plan válido. Para cumplir con los requisitos del estudio, se estableció un límite de tiempo estricto (timeout) de 1 minuto (60 segundos) por instancia. Si el planificador no logra devolver una solución en ese intervalo, el script detiene la ejecución y determina el límite máximo de complejidad alcanzado

Análisis de Resultados y Rendimiento del Planificador FF

Tras la ejecución del banco de pruebas, se generó una gráfica cruzando el tamaño del problema con el tiempo requerido para encontrar la solución

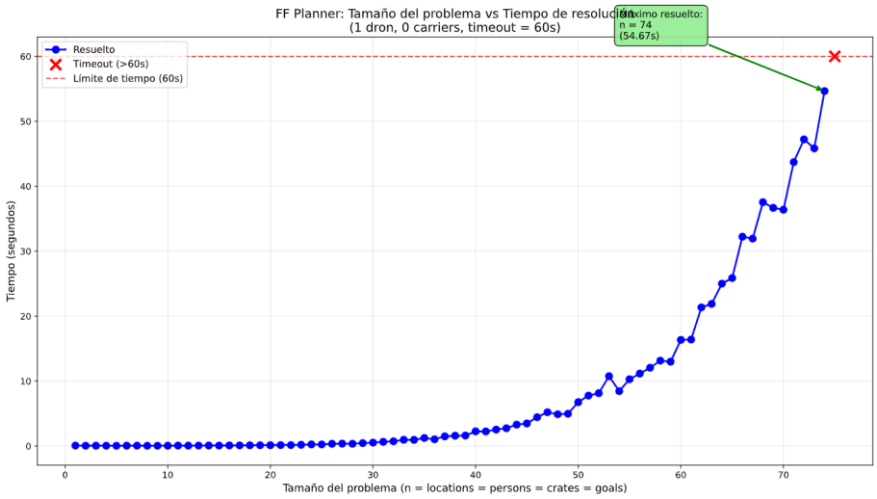

Figura 1.2.4 Gráfica tiempos de ejecución para cada tamaño de problema

Analizando los datos obtenidos, el planificador FF demostró una notable capacidad de resolución, logrando resolver un problema de un tamaño máximo de n=74 dentro del tiempo límite estipulado, empleando para ello un total de 54,67 segundos en su última ejecución exitosa.

Al observar el comportamiento de la curva, se pueden destacar las siguientes fases y características fundamentales:

- Fase de coste constante ($n = 1$ hasta $n \approx 30$): En los problemas de tamaño inicial y medio, el tiempo de resolución es prácticamente insignificante, manteniéndose muy cercano a los 0 segundos. Esto indica que, para un número reducido de variables, la heurística de FF es capaz de guiar la búsqueda directamente hacia la meta sin apenas explorar ramificaciones innecesarias.
- Crecimiento no lineal ($n > 30$): A partir del tamaño 30, la gráfica muestra una tendencia de crecimiento empinado que refleja la explosión combinatoria del espacio de estados. Al aumentar simultáneamente el número de refugios, personas, cajas y metas, la cantidad de combinaciones posibles de emparejamiento entre cajas y personas (y las rutas del dron para lograrlas) crece enormemente.
- Fluctuaciones heurísticas: A lo largo de la fase de crecimiento (por ejemplo, entre n=50 y n=70), se aprecian picos y valles en el tiempo de resolución.

Al generarse las ubicaciones y necesidades de forma aleatoria, algunas instancias del problema presentan "mesetas" en el espacio de búsqueda que le cuestan más tiempo superar al algoritmo, mientras que en otros tamaños mayores la disposición aleatoria favorece que el planificador encuentre un atajo directo de forma más veloz.

Para resumir, el planificador FF se muestra sumamente eficiente en este dominio STRIPS, aunque sucumbe ante la explosión combinatoria alrededor del tamaño 72 cuando el coste computacional excede el límite temporal de un minuto.

EJERCICIO 1.3: Comparativa rendimiento algoritmos de búsqueda y heurísticas

Para la resolución de este apartado se lleva a cabo un proceso de tres pasos:

Primer Apartado:

- Como primera tarea, se pide a través de problemas de tamaño creciente investigar el problema de mayor tamaño que pueden resolver los algoritmos Breadth-First Search (BFS), Iterative Deepening Search (IDS), A-Star (A*) y Greedy Best-First Search (GBFS) en un tiempo de 60 segundos y donde se debe aplicar la heurística hMAX sobre A* y GBFS.

  Para la generación de los problemas se utiliza un script de Python que genera 30 problemas donde de forma aleatoria se determinan los contenidos de las cajas, las ubicaciones de las personas y las necesidades de las personas.

  A partir de los problemas generado se han obtenido los siguientes resultados para cada uno de los algoritmos:

Algoritmo	Max Tamaño	Tiempo (s)	Acciones Plan	Óptimo
BFS	5	6.031	15	Sí (coste uniforme)
IDS	4	23.345	11	Sí (coste uniforme)
A* + hMAX	5	32.283	15	Sí (hMAX admisible)
GBFS +hMAX	7	52.415	25	No

Tabla 1.3.1 Problema de mayor tamaño resuelto en 60 segundos para cada algoritmo

  Las conclusiones que se pueden extraer de cada uno de los resultados son las siguientes:

  - BFS: Debido a su naturaleza de búsqueda expansiva a través de todos los diferentes niveles y a que el coste de cada movimiento es de 1, es capaz de encontrar la solución óptima en un margen de tiempo más corto que por ejemplo A* +hMAX.

    Esto se debe a que BFS almacena cada uno de los nodos por los que pasa de forma que observa todos los caminos posibles a la vez y no se queda atrapado en ramas que no conducen a ningún lugar, siendo por ello su mayor limitante la memoria disponible

- IDS: Su mayor limitante es la memoria RAM disponible pues se repite la búsqueda en los niveles anteriores de forma que, aunque mantenga la optimalidad tiene como contrapartida un gran coste computacional.

  - A* + hMAX: A pesar de que A* es un algoritmo óptimo, al tener que realizar cálculos heurísticos de hMAX el tiempo de ejecución del algoritmo se ve incrementado frente a otros como el de BFS, el cual encuentra la misma solución en unos 25 segundos de diferencia.

- GBFS + hMAX: Debido a la naturaleza voraz de GBFS, no se tiene que preocupar por encontrar la solución óptima lo que le permite llegara resolver problemas de mayor

tamaño que los anteriores algoritmos no pudieron. Además, con la ayuda de hMAX le permite ir directamente al camino que le propone la heurística que debe seguir para alcanzar la meta a pesar de que es el camino de mayor coste.

Segundo Apartado

1. Como segunda tarea se pidió que a partir del anterior problema de mayor tamaño resuelto por GBFS se aplicaran todas las posibles combinaciones de GBFS y EHC con las heurísticas hMAX, hADD, hFF y LandMark. A partir de la ejecución se obtuvieron los siguientes resultados:

Algoritmo + Heurística	Tiempo (s)	Acciones Plan
GBFS+hMAX	51.675	25
GBFS+hADD	0.37	26
GBFS+hFF	0.247	23
GBFS+Landmark	0.108	27
EHC+hMAX	TIMEOUT	-
EHC+hADD	1.0141	29
EHC+hFF	0.502	26
EHC+Landmark	0.124	31

Tabla 1.3.2 Tiempo y nº de acciones para GBFS y EHC con cada heurística para un problema de tamaño 7

La primera conclusión que se puede extraer es lo ineficiente que demuestra ser la heurística hMAX frente a las demás debido a la gran cantidad de tiempo que tarda en resolver el problema, no siendo incluso capaz de resolverlo con EHC.

Como segunda conclusión posible se observa que la ausencia de memoria (backtracking) por parte de EHC le lleva a proponer soluciones peores que las de GBFS.

Por último, se puede observar como las mejores heurísticas son la de Landmark y la de hFF, siendo la primera la más rápida con diferencia y la segunda la que encuentra el mejor camino en un poco más de tiempo. De esta forma en función de si se necesitan unos resultados en tiempo real, la heurística Landmark sería la mejor opción, mientras que, si se quisiera encontrar el mejor plan para reducir los costes a costa de incrementar el tiempo de cálculo, hFF sería la mejor opción. hADD a pesar de ser también rápida, al sumar todo tiende a sobreestimar.

Tercer Apartado

2. Por último, se pidió resolver para BFS, IDF y A* con cada heurística admisible el problema de mayor tamaño resuelto por A* con hMAX. Los resultados que se obtuvieron fueron los siguientes:

Algoritmo	Tiempo (s)	Acciones Plan	Solución Óptima
BFS	6.169	15	Sí
IDS	TIMEOUT	-	-

A*+hMAX	32.214	15	Sí
A*+lmcut	26.26	15	Sí

Tabla 1.3.3 Tiempo y nº de acciones para BFS, IDS, A*+hMax y A*+lmcut para un problema de tamaño 5

Las conclusiones que se pueden extraer de BFS y A*+hMAX son las mismas que ya se han comentado en la primera tarea de este ejercicio. Además, para IDS el resultado de TIMEOUT es consecuente, puesto que en la primera tarea se observó que el tamaño de problema máximo que fue capaz de resolver fue de tamaño 4, siendo por ello razonable que no fuera capaz de resolver un problema de tamaño 5.

Por último, se puede observar que entre los algoritmos A* (garantizan una solución óptima) la mejor heurística admisible es la de Landmark Cut, siendo capaz de resolver el mismo problema es 6 segundos menos.

PARTE 2: PLANIFICACIÓN CON NÚMEROS Y OPTIMIZACIÓN

EJERCICIO 2.1: SERVICIO DE EMERGENCIAS, TRANSPORTADORES

En la versión inicial del dominio, el dron disponía de dos brazos (grip) para transportar las cajas. Sin embargo, escalar esta solución aumentando el número de brazos para incrementar la capacidad de carga generaba un grave problema de explosión combinatoria, ya que el planificador distinguía entre estados simétricos (p. ej., caja A en el brazo izquierdo y caja B en el derecho, frente a la disposición inversa).

Para solucionar esto, en este ejercicio se ha eliminado por completo el tipo grip y los brazos del dron, reduciendo su capacidad de agarre simultáneo a una sola caja mediante un brazo implícito controlado con el predicado (free ?dron). Como contrapartida, se ha introducido el objeto carrier (transportador) con capacidad para 4 cajas.

Para evitar caer en el mismo error de explosión de estados utilizando "ranuras" específicas dentro del transportador, la capacidad se ha modelado de forma genérica mediante predicados matemáticos. Se ha definido un nuevo tipo num con cinco objetos discretos (n0, n1, n2, n3, n4) y un predicado relacional (siguiente ?n1 - num ?n2 - num). De este modo, el estado del transportador simplemente lleva la cuenta del número total de cajas que contiene mediante el predicado (boxes-in-carrier ?c ?n), sin importar el orden interno de las mismas.

Ahora bien, la incorporación del transportador ha requerido una reestructuración de las acciones del dominio:

- Carga y descarga: Se han añadido las acciones put-in-carrier y take-from-carrier. Estas comprueban que la capacidad no exceda los límites sumando o restando una unidad matemática a través del predicado siguiente.
- Movilidad estricta: Se ha implementado la acción move-carrier para desplazar el dron junto con el transportador. Además, se ha tomado la decisión de diseño de incluir el predicado (free ?dron) como precondición obligatoria tanto en move como en move-carrier. Esto impide lógicamente que el dron pueda volar llevando una caja directamente en su agarre, forzando al planificador a utilizar el transportador de manera realista para cualquier traslado entre localizaciones.

Una vez explicado los cambios sobre la parte 1, podemos pasar a evaluar el impacto de este nuevo dominio sobre el espacio de búsqueda, se han replicado las pruebas de la Parte 1 empleando pyperplan.

Los resultados obtenidos para los planificadores de satisfacción (ejecutados sobre un problema de tamaño 5) son los siguientes:

Algoritmo + Heurística	Tiempo (s)	Acciones del Plan
GBFS + hMAX	32.988	23
GBFS + hADD	0.597	28
GBFS + hFF	0.530	29
GBFS + Landmark	0.376	29
EHC + hMAX	TIMEOUT	-
EHC + hADD	1.209	35
EHC + hFF	2.463	31
EHC + Landmark	0.350	32

Tabla 2.1.1 Tiempo y nº de acciones para GBFS y EHC con cada heurística para el problema de tamaño máximo de GBFS con el nuevo dominio

Los resultados obtenidos para los **planificadores óptimos** (ejecutados sobre un problema de tamaño 4) son:

Algoritmo	Tiempo (s)	Acciones del Plan	Solución Óptima
BFS	2.514	19	Sí
IDS	TIMEOUT	-	-
A* + hMAX	39.340	19	Sí
A* + lmcut	TIMEOUT	-	-

- Tabla 2.1.1 Tiempo y nº de acciones para BFS, IDS, A*+hMax y A*+lmcut para el problema de tamaño máximo de A*+hmax con el nuevo dominio

Con estos resultados podemos destacar varias cosas:

- Entregar una caja requiere ahora obligatoriamente una cadena de acciones más larga para el mismo objetivo (`pick` -> `put-in-carrier` -> `move-carrier` -> `take-from-carrier` -> `leave`) por lo que la longitud del plan ha aumentado en la mayotia de los casos. El plan óptimo para un tamaño 4 requiere ahora 19 acciones, un aumento respecto al modelo sin transportadores que era de 15.
- se aprecia un aumento de la ramificación ya que la inclusión de nuevas acciones y la gestión numérica ha encarecido notablemente la búsqueda. Esto se evidencia de forma drástica en el planificador *A + lmcut**, el cual en la Parte 1 era la mejor heurística admisible y aquí agota el tiempo (TIMEOUT) sin poder resolver un problema de tamaño 4.
- Al aplicar EHC con hMAX, el planificador no logra encontrar solución a tiempo (`TIMEOUT`). La falta de *backtracking* en EHC hace que se quede atrapado en mesetas por los pasos intermedios de carga y descarga del transportador, donde la heurística hMAX no es lo suficientemente informativa para indicarle una mejora inmediata. GBFS, al evaluar globalmente la frontera de búsqueda, sí logra sortear estas mesetas (tardando 32.9 segundos).
- Eficiencia de Landmark y hFF: A pesar de la mayor complejidad del árbol de estados, heurísticas como `Landmark` demuestran una excelente adaptabilidad. En algoritmos voraces logra encontrar soluciones en apenas ~0.35 segundos, aunque a costa de generar planes mucho más largos y subóptimos (hasta 32 acciones con EHC).

EJERCICIO 2.2: SERVICIO DE EMERGENCIAS, COSTES DE ACCIÓN

Para la realización de este ejercicio, es necesario adaptar el dominio del apartado anterior para añadir los costes de las diferentes acciones. Con esto, sumado a la adición del transportador, se trata de hacer que el dron lleve más de una caja por viaje, teniendo en cuenta las consecuencias de volver al depósito sin llevar más de una caja.

Para ello, se añaden como funciones el total-cost y el coste del vuelo desde una localización a otra:

```
(:functions
    (total-cost) 6↗ - number
    (fly-cost ?l1 ?l2 - location) 2👁 - number
)
```

Figura 2.2.1 Funciones añadidas para añadir el coste a las acciones

La función fly-cost se empleará para añadir el coste de los vuelos del dron (y carrier) entre las diferentes localizaciones al coste total, mientras que el resto de las acciones incrementarán este valor en 1.

Además, para el correcto funcionamiento de los problemas con este dominio, es necesario modificar el generador de problemas, de tal modo que, siguiendo con la aleatoriedad que se busca en los diversos problemas para el estudio, se asignen costes aleatorios (desde 1 hasta 20) entre cada una de las ubicaciones (diferentes también entre la ida y la vuelta). Siempre se partirá de inicio con un coste total de 0.

```
# --- Costes y Funciones ---
f.write("\t(= (total-cost) 0)\n")
for l1 in locations:
    for l2 in locations:
        if l1 != l2:
            cost = random.randint(1, 20)
            f.write(f"\t(= (fly-cost {l1} {l2}) {cost})\n")
```

Figura 2.2.2 Fragmento de código del generador de problemas encargado de asignar los costes

Este generador creará 50 problemas de tamaño diferente. En cuanto a tamaño, se refiere a cantidad de personas, cajas y localizaciones (excluyendo el depósito). En la figura 2.2.3, se puede observar un problema de tamaño 1, cumpliendo con las restricciones dadas en el enunciado de un único con dron con un solo transportador de tamaño 4.

```
(define (problem problem_size1)
(:domain emergencias-costs)
(:objects
    dron1 - dron
    deposito refugio1 - location
    box1 - box
    comida medicina - bcontent
    person1 - person
    carrier1 - carrier
    n0 n1 n2 n3 n4 - num
)

(:init
    (= (total-cost) 0)
    (= (fly-cost deposito refugio1) 19)
    (= (fly-cost refugio1 deposito) 18)
    (siguiente n0 n1)
    (siguiente n1 n2)
    (siguiente n2 n3)
    (siguiente n3 n4)
    (at-dron dron1 deposito)
    (free dron1)
    (at-carrier carrier1 deposito)
    (boxes-in-carrier carrier1 n0)
    (at-box box1 deposito)
    (box-has box1 medicina)
    (at-person person1 refugio1)
)

(:goal (and
    (person-has person1 medicina)
))
(:metric minimize (total-cost))
)
```

Figura 2.2.5 Problema de tamaño 1

Una vez generados los problemas, se ha procedido a generar un benchmark encargado de recoger los resultados obtenidos por diversos planificadores, con relación al tamaño máximo del problema que son capaces de resolver en menos de un minuto y el coste de este, para su posterior comparación.

Estudio de resultados para planificadores de satisfacción

Se ha desarrollado una tabla con los resultados de los diversos planificadores de satisfacción estudiados (Tabla 2.2.1).

Planificadores	Tamaño máximo	Coste total
metric-ff	11	290
lama-first	30	679
seq-sat-fdss-2	28	406
seq-sat-fd-autotune-2	8	125

Tabla 2.2.1 Resultados obtenidos de los planificadores de satisfacción

i. LAMA-first (Tamaño 30 - Coste 679): Es el planificador más robusto y escalable. Esta escalabilidad se debe a la Heurística de Landmarks (hitos intermedios). En lugar de ver el problema como un todo, identifica estados obligatorios por los que debe pasar (por ejemplo, la caja debe estar en el transportador). Esto le permite "trocear" la búsqueda y resolver problemas de gran tamaño, aunque el coste final suele ser el más elevado debido a su naturaleza.

- seq-sat-fdss-2 (Tamaño 28 - Coste 406): Este planificador ejecuta una serie de algoritmos distintos de forma secuencial (porfolio secuencial), es decir, uno tras otro repartiendo el tiempo. Es el más equilibrado: llega casi al mismo tamaño que LAMA pero con un coste mucho menor (406 frente a 679). Esto demuestra que combinar diferentes estrategias de búsqueda permite encontrar rutas mucho más eficientes con el transportador. Se debe tener en cuenta que el coste puede verse reducido por el tamaño del problema.

ii. metric-ff (Tamaño 11 - Coste 290): Está diseñado para manejar valores numéricos directamente. Utiliza una técnica llamada Enforced Hill-Climbing (Escalada de Colinas), que busca siempre el paso más directo a la meta. Es muy rápido en problemas pequeños, pero en el tamaño 12 se detiene. Esto ocurre porque, si no encuentra una mejora inmediata en la dirección que lleva, el algoritmo se rinde en lugar de seguir buscando, por eso no llega a agotar el minuto de tiempo.

iii. seq-sat-fd-autotune-2 (Tamaño 8 - Coste 125): Es un sistema que intenta auto-ajustarse basándose en problemas de competiciones anteriores. Su bajo rendimiento en este dominio (Size 8) sugiere que las configuraciones que tiene "aprendidas" no se adaptan bien a la lógica del transportador y los costes de vuelo aleatorios.

Estudio de resultados para planificadores de optimización

Se ha desarrollado una tabla con los resultados de los diversos planificadores de optimización estudiados (Tabla 2.2.2).

Planificadores	Tamaño máximo	Coste total
seq-opt-lmcut	5	34
seq-opt-bjolp	6	48
seq-opt-fdss-2	6	48

Tabla 2.2.2 Resultados obtenidos de los planificadores de optimización

iv. seq-opt-lmcut (Tamaño: 5 – Coste: 34): Utiliza la heurística Landmark Cut. Es una técnica muy precisa para obtener estimaciones de coste, pero resulta costosa de calcular por cada nodo explorado. En este dominio se ha detenido un paso antes que los demás, priorizando la precisión de la búsqueda sobre la velocidad de exploración.

v. seq-opt-bjolp (Tamaño: 6 – Coste: 48): Se basa en el Big Joint Landmark Optimal Planning. Es un planificador especializado que extrae de forma exhaustiva hitos u objetivos intermedios (landmarks) para construir una heurística muy potente. Ha demostrado eficiencia en este dominio al alcanzar el tamaño 6.

- seq-opt-fdss-2 (Tamaño: 6 – Coste: 48): Es un porfolio secuencial ("Stone Soup"). A diferencia de los anteriores, no es un solo algoritmo, sino que reparte el tiempo entre diferentes estrategias óptimas. Al alcanzar el tamaño 6 demuestra la versatilidad de combinar varios algoritmos para adaptarse a la estructura del problema.

Los resultados validan el compromiso entre calidad y tiempo: mientras que los planificadores de satisfacción (como LAMA) pueden resolver problemas de tamaño 30 aceptando costes más altos, los óptimos se ven restringidos a tamaños muy reducidos debido al enorme esfuerzo computacional que exige garantizar el mínimo coste.

PARTE 3: PLANIFICACIÓN CON TIEMPOS

Para llevar a cabo la parte 3 de la práctica primero se tuvieron que llevar a cabo modificaciones del dominio de la parte 2.2, siendo las modificaciones más relevantes las siguientes:

Se eliminan los *actions-costs* para sustituirlos por las *durative-actions*, debido a que ya no se optimiza el coste acumulado si no el tiempo total. Además, se sustituye el *total-cost* por el empleo de un *metric minimize (total-time)*, pues como se ha comentado el objetivo es minimizar el tiempo de vuelo.

Para poder asignar tiempos a las acciones es importante añadir a cada una de las acciones el tiempo que toma para llevarlas a cabo, donde los de movimiento del dron y del transportador serán decididos en tiempo de ejecución, mientras que las demás acciones reciben un valor fijo de duración de 5 segundos.

Por último, para que la concurrencia sea posible, se emplean condicionantes y efectos que se aplican al comienzo *(at start)* o al final *(at end),* actuando como cerrojos.

Una vez aplicados todos estos cambios, se pidió investigar el problema de tamaño máximo que era capaz de resolver el planificador Optic en un margen de 60 segundos, aumentando de forma secuencial el número de drones y transportadores a la vez, donde antes de pasar al siguiente nivel, se debía incrementar uno a uno el tamaño de la meta.

El funcionamiento del planificador Optic es el siguiente: Emplea un modo llamado *anytime* donde primero busca una solución rápidamente y a continuación continúa buscando soluciones con menor o igual duración de tiempo hasta que no es capaz de encontrar una sin agotar el timeout. Aquellas ramas expandidas y podadas por superar la duración de la mejor solución encontrada hasta el momento no vuelven a ser consideradas en sucesivas búsquedas.

Una vez sentadas todas las bases necesarias para llevar a cabo el ejercicio, se obtuvieron los siguientes resultados:

Drones+ Transport	Goals	Nº Sol	Pasos 1ª	Duración 1ª	Tiempo 1ª	Pasos Ult	Duración Ult	Tiempo Ult
1	1	3	5	38	0.04	6	30	2.22
1	2	3	11	53	0.27	9	43	49.96
1	3	4	15	88	0.35	14	64	44.44
1	4	2	23	143	1.15	24	137	28.02
1	5	2	27	179	3.72	25	160	47.99
2	1	4	9	77	2.3	9	30	5.27
2	2	1	12	45	5.86	12	45	5.86
3	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
4	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
5	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A

Tabla 3.1.1 Resultados de la ejecución del planificador Optic

A partir de los resultados se puede observar cómo Optic se centra en mejorar la solución minimizando el tiempo, no el número de pasos, ya que, para 1 dron y 1 transportador, la solución última incluye un paso más que la primera solución, pero minimiza el tiempo en 8 segundos.

Además, para problemas con mayor número de metas, la optimización es más notable, por ejemplo, para tres metas donde se pasa de 88 segundos a 64 segundos. Sin embargo, a medida que aumenta la complejidad del problema cuesta más estar dentro del margen de tiempo de 60 segundos, puesto que se aumenta el factor de ramificación, aumentando el espacio de búsqueda y llevando al planificador a explorar un número mayor de combinaciones sin encontrar una rama que termine satisfaciendo la meta.

CONCLUSIONES

El análisis demuestra que el planificador FF es altamente eficiente en dominios clásicos, resolviendo problemas de hasta tamaño n=72 antes de verse limitado por la explosión combinatoria. Se identificó un claro compromiso entre calidad y tiempo: mientras los planificadores de satisfacción como LAMA escalan a problemas mayores mediante el uso de hitos (landmarks), los de optimización quedan restringidos a tamaños muy reducidos por el alto coste de garantizar la solución mínima. Finalmente, el uso de Optic confirmó que la concurrencia permite optimizar el tiempo total de forma más efectiva que el simple recuento de pasos, aunque la complejidad aumenta drásticamente al añadir más drones y metas.